

1. Core Architecture & "Safe Zone" Gateway

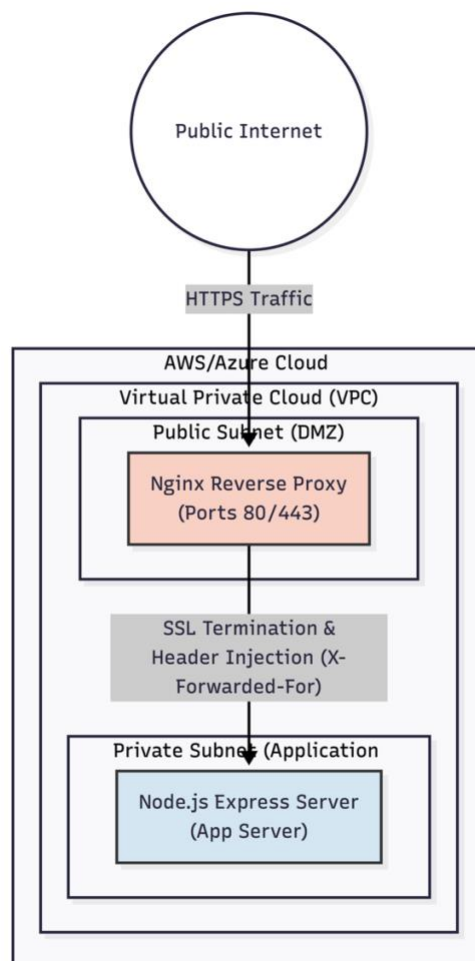
Lead Engineer: Sagiv Levy

Domain: Cloud Infrastructure, Traffic Routing Algorithms, and Legitimate Application Integrity.

1.1. Cloud Network Topology & Reverse Proxy

The infrastructure is built on a high-availability cloud model (NGrok). The application is shielded from the public internet using a Reverse Proxy architecture.

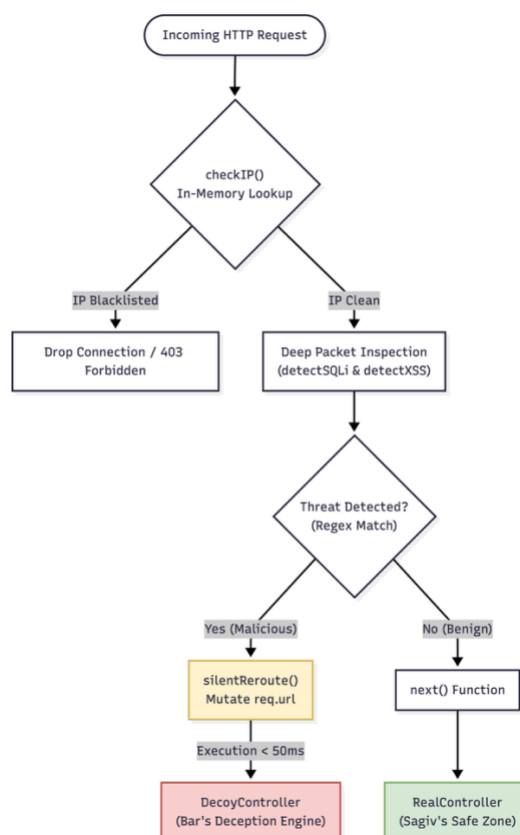
- **Edge Routing (Nginx):** Nginx serves as the gateway, bound to ports 80 (HTTP) and 443 (HTTPS). It is configured for SSL/TLS termination, decrypting incoming traffic to offload cryptographic overhead from the Node.js event loop.
- **Header Injection:** To preserve the attacker's true origin, Nginx intercepts the connection and injects the X-Forwarded-For and X-Real-IP headers before routing the payload to the backend Express server.



1.2. The "Gatekeeper" Traffic Middleware

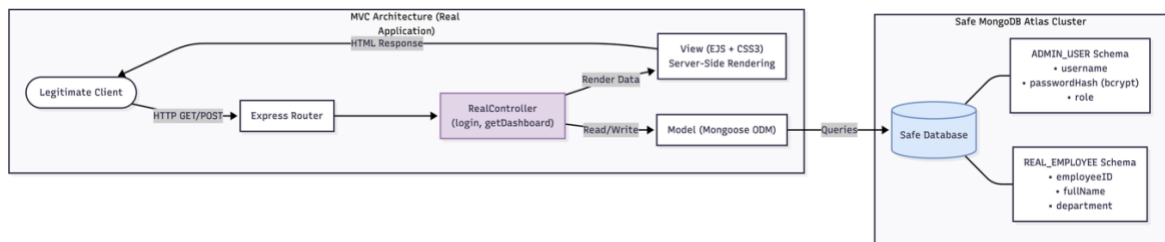
This is a custom-built, highly optimized Express middleware pipeline designed to filter traffic with near-zero overhead.

- **IP Reputation (checkIP):** Extracts the origin IP and performs an $O(1)$ lookup against an in-memory cached blacklist.
- **Deep Packet Inspection (detectSQLi & detectXSS):** Utilizes non-blocking Regular Expressions (Regex) to scan req.body and req.query. All regex patterns are strictly audited to prevent ReDoS (Regular Expression Denial of Service) vulnerabilities, ensuring attackers cannot crash the gateway with catastrophic backtracking.
- **The silentReroute Algorithm:** If a threat is detected, the middleware **does not** issue an HTTP 302 redirect. Instead, it internally mutates the req.url property to point to the Decoy Controller and invokes Express's next() function. This creates a seamless, invisible handoff that must execute in under **50ms** to avoid alerting the attacker.



1.3. Legitimate Application ("Safe Zone")

- **MVC Architecture:** The RealController handles benign traffic via standard Express routes (renderLandingPage, login, getDashboard).
- **Server-Side Rendering:** Uses EJS (Embedded JavaScript) and CSS3 to render the corporate UI on the server, ensuring fast Time-to-First-Byte (TTFB) and hiding application logic from client-side scraping.
- **Secure Persistence:** MongoDB Atlas with Mongoose ODM. Schemas (ADMIN_USER, REAL_EMPLOYEE) enforce strict data typing. Passwords are mathematically secured using the bcrypt library with a high salt work factor prior to insertion.



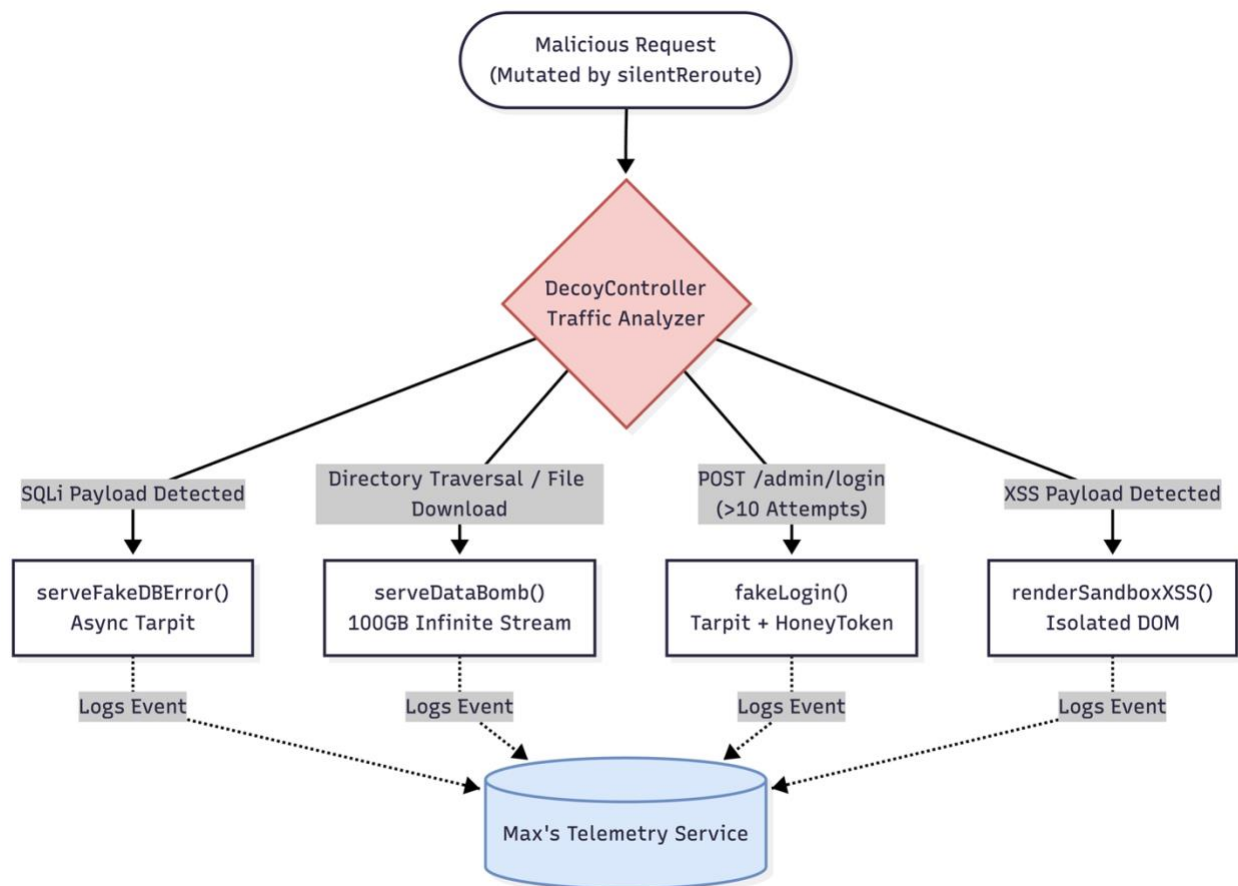
2. Deception Engine & Traps

Lead Engineer: Bar

Domain: Honeypot Logic, Attacker Resource Exhaustion, and Dynamic Decoy Generation.

2.1. The Decoy Controller & Routing Logic

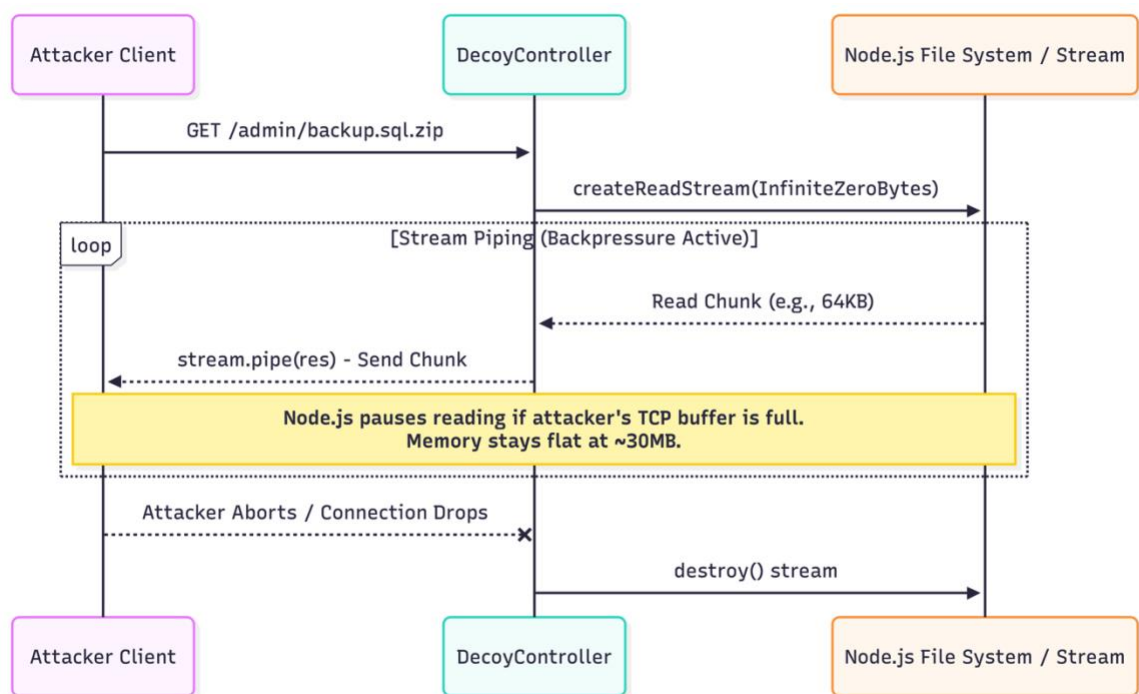
The DecoyController acts as a parallel universe to the Safe Zone. It catches all mutated request objects handed over by Sagiv's middleware.



2.2. Trap Implementation & Memory Management

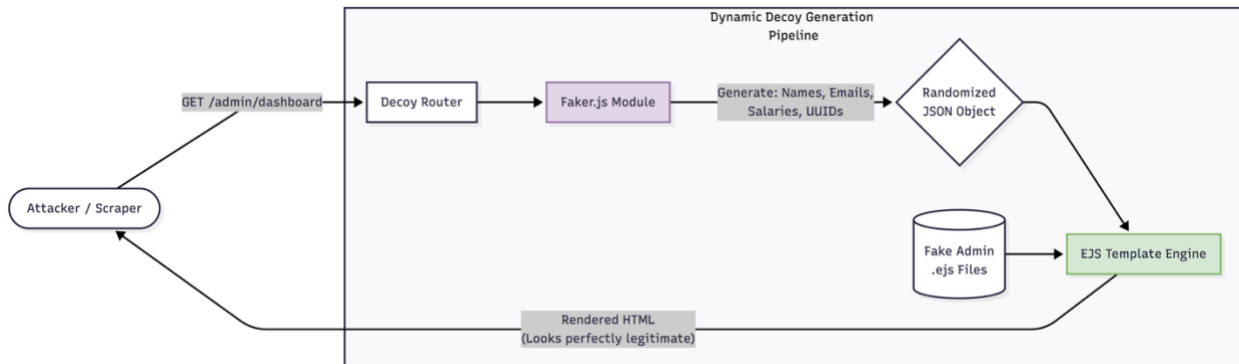
The core objective is to waste the attacker's resources (CPU, Bandwidth, Time) without compromising the host machine's stability.

- **The "Data Bomb" (serveDataBomb):** Streams 100GB of zero-byte or garbage data to the attacker. To prevent the Node.js server from running out of RAM, this is implemented using native Node.js fs and Stream APIs. The stream leverages **backpressure handling** (stream.pipe(res)), ensuring data is only pushed as fast as the attacker's network can consume it.
- **The Tarpit (serveFakeDBError):** Simulates a vulnerable database taking a long time to respond. Implemented using asynchronous setTimeout functions wrapped in JavaScript Promises. This artificially holds the HTTP connection open, exhausting the attacker's active thread pool while allowing the non-blocking Node.js event loop to freely serve other legitimate requests.
- **Logic Traps:** Includes fakeLogin (simulating delayed 10th-attempt brute-force lockouts), serveHoneyToken (providing trackable fake credentials), and renderSandboxXSS (a safe DOM environment that logs XSS payloads).



2.3. Dynamic Decoy Generation

- **fakerLogin.js Integration:** The fake "InnoTech Admin" dashboard is never static. On every GET request, the server uses the fakerLogin.js module to dynamically seed EJS templates with randomized, realistic data (employee names, believable corporate emails, financial salary data). This ensures that automated scraping tools see constantly changing data, keeping the attacker engaged.



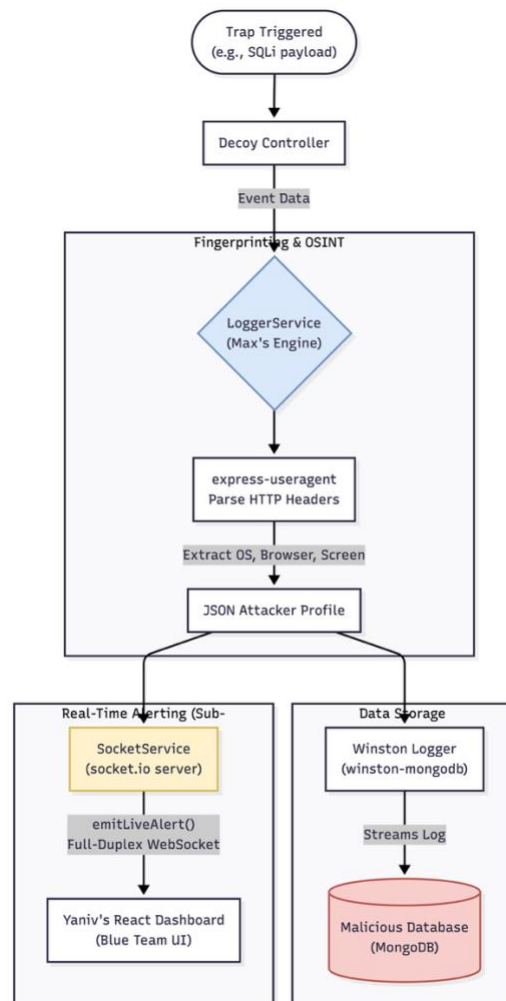
3. Telemetry, Logging & Data Extraction

Lead Engineer: Max

Domain: Threat Intelligence, Real-time Data Piping, and Air-Gapped Database Management.

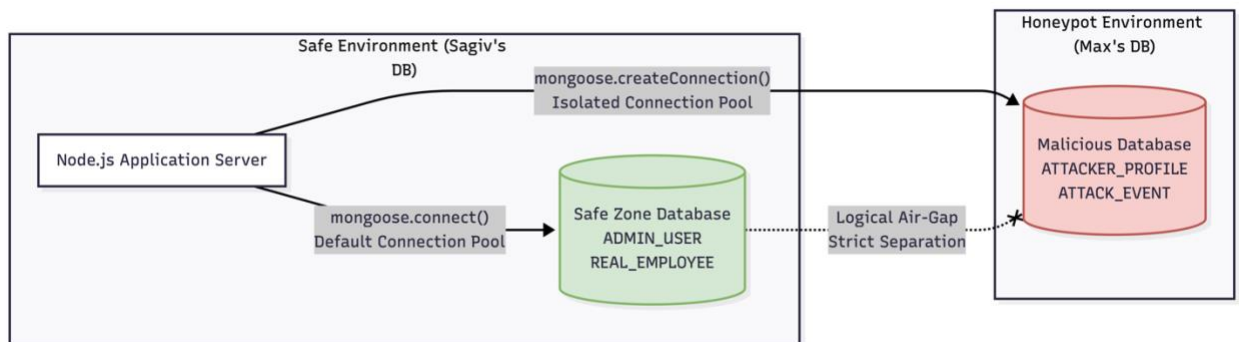
3.1. Fingerprinting & OSINT Collection

- **Header Analysis:** Integration of the express-useragent module. The system parses the raw HTTP headers to extract detailed metadata: Operating System, browser version, device type. This data is structured into a JSON profile for every unique attacker.



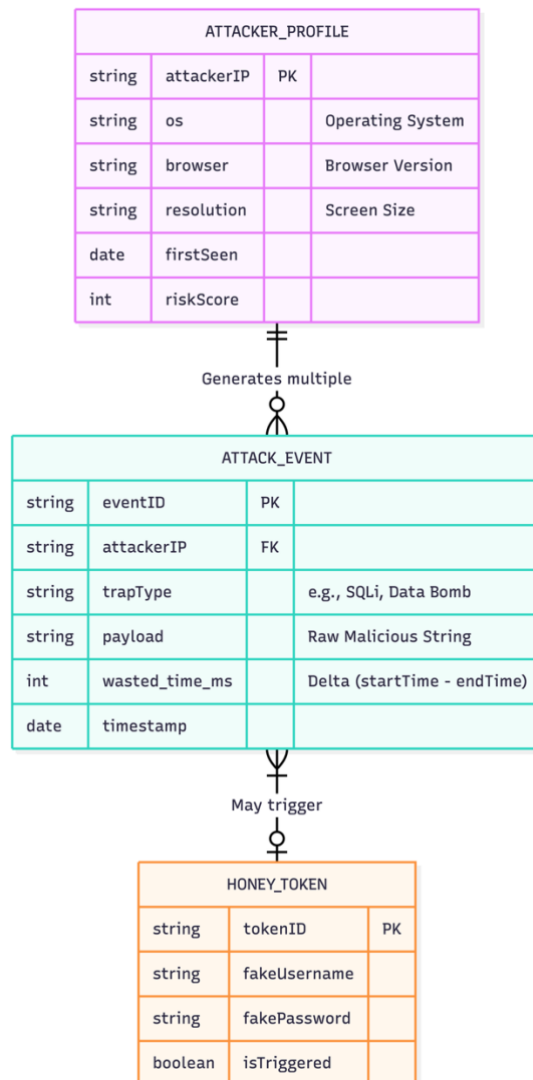
3.2. Real-Time Telemetry Pipeline

- **Winston Logging Engine:** The LoggerService uses Winston configured with the winston-mongodb transport layer. It asynchronously streams structured log data directly to the database. It calculates sophisticated metrics, such as wasted_time_ms, by storing a timestamp upon trap entry and calculating the delta upon connection termination.
- **WebSocket Infrastructure (SocketService):** Implements server-side socket.io. The moment a trap is triggered (e.g., an SQLi payload is fired), the Decoy Controller triggers an event. The Socket Service immediately broadcasts an emitLiveAlert JSON payload via a full-duplex WebSocket channel to the connected Blue Team dashboard, achieving sub-second latency.



3.3. Isolated "Malicious" Database Architecture

- **Logical Air-Gapping:** To guarantee that an attacker cannot pivot from the honeypot into the safe system, Max establishes an entirely separate Mongoose connection pool using `mongoose.createConnection()`.
- **Malicious Schemas:** Dedicated schemas are defined for ATTACKER_PROFILE, ATTACK_EVENT, and HONEY_TOKEN. This physical and logical separation ensures zero cross-contamination with Sagiv's ADMIN_USER data.



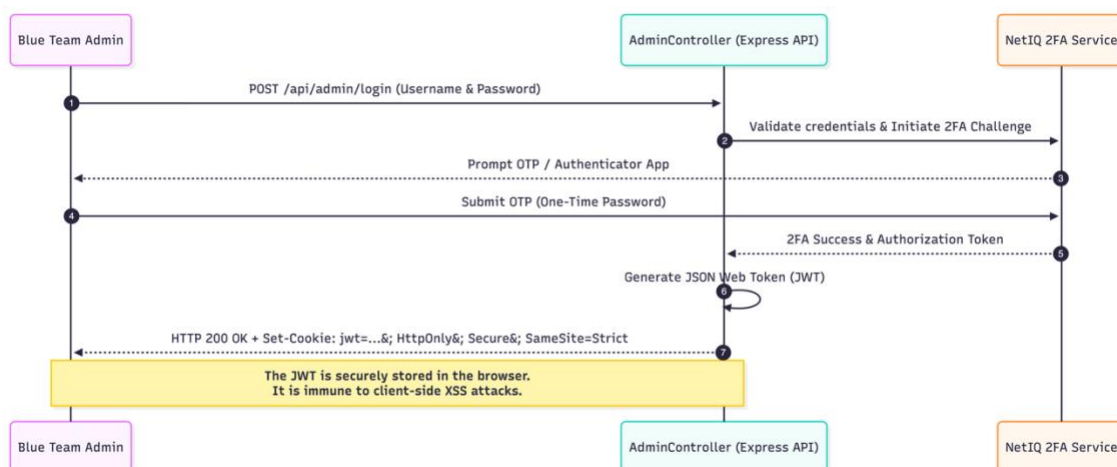
4. Admin Dashboard, API & Security (The "Blue Team" Interface)

Lead Engineer: Yaniv

Domain: Secure REST APIs, React SPA, and Geographical Threat Visualization.

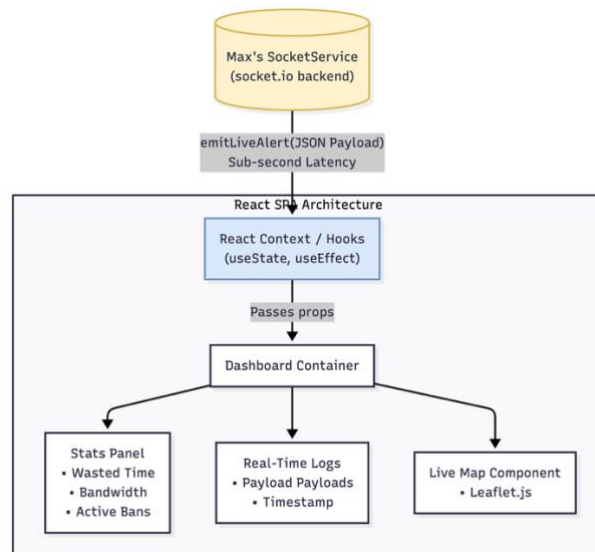
4.1. Secure Administrative API

- **Authentication & 2FA:** The AdminController orchestrates the login process. To protect the Blue Team dashboard, NetIQ SDK / OAuth is integrated for Two-Factor Authentication (2FA).
- **JWT Lifecycle:** Upon successful 2FA, the server issues a JSON Web Token (JWT). To prevent Cross-Site Scripting (XSS) from stealing the token, the JWT is strictly stored in an HttpOnly, Secure cookie, validating authorization for protected endpoints (/admin/map, /admin/ban).



4.2. React SPA & Real-Time State Management

- **Component Architecture:** The frontend is a Single Page Application (SPA) built with React.js and React Router. The UI adheres to a strict Teal/Orange cybersecurity color palette.
- **Socket Integration:** The socket.io-client library is bound directly to React's state management (e.g., useState and useEffect hooks). When an emitLiveAlert is received from Max's backend, the State updates instantly, causing the REAL-TIME LOGS and STATS panels (Wasted Time, Bandwidth, Active Bans) to re-render dynamically without a page refresh.



4.3. Geographic Visualization (Live Map)

- **GeoIP & Mapping:** When a socket event fires, the frontend extracts the attacker's IP. It queries a GeoIP lookup service (e.g., IP-API) to translate the IP into global coordinates (Latitude/Longitude).
- **Visualizing the Threat:** Using Leaflet.js (or react-leaflet), the system plots the data dynamically. The origin of the attack and our server location are rendered as (Nodes) on the map. The active connection and attack trajectory between them are visualized by drawing connecting (Edges). This provides the Blue Team with an immediate, visual understanding of global attack vectors.

